

```
-- 临时创建 2 个字段的表结构
SQL > create table emp_tmp select emp_no,hire_date from emp;
SQL > desc emp_tmp;
+-----+-----+-----+-----+-----+-----+
| Field      | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_no     | int  | NO   |     | NULL    |       |
| hire_date  | date | NO   |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+

-- 导入数据语句
load data infile '/data/mysql/3306/tmp/employees.txt'
replace into table demo.emp_tmp
character set utf8mb4
fields terminated by ','
enclosed by '"'
lines terminated by '\n'
(@C1,@C2,@C3,@C4,@C5,@C6) -- 该部分对应 employees.txt 文件中 6 列数据
-- 只对导出数据中指定的 2 个列与表中字段做匹配,mapping 关系指定的顺序不影响导入结果
set hire_date=@C6,
    emp_no=@C1;

-- 导入数据结果示例
SQL > select * from emp_tmp;
+-----+-----+
| emp_no | hire_date |
+-----+-----+
| 10001  | 1986-06-26 |
| 10002  | 1985-11-21 |
| 10003  | 1986-08-28 |
| 10004  | 1986-12-01 |
| 10005  | 1989-09-12 |
| 10006  | 1989-06-02 |
| 10007  | 1989-02-10 |
| 10008  | 1994-09-15 |
| 10009  | 1985-02-18 |
| 10010  | 1989-08-24 |
+-----+-----+
10 rows in set (0.0016 sec)
```

场景 2. LOAD 文件中的字段比数据表中的字段少

表字段不仅包含文本文件中所有数据，还包含了额外的字段

```
-- 导入数据语句
load data infile '/data/mysql/3306/tmp/employees.txt'
replace into table demo.emp
character set utf8mb4
fields terminated by ','
enclosed by '"'
lines terminated by '\n'
(@C1,@C2,@C3,@C4,@C5,@C6) -- 该部分对应 employees.txt 文件中 6 列数据
-- 将文件中的字段与表中字段做 mapping 对应,表中多出的字段不做处理
set emp_no=@C1,
    birth_date=@C2,
    first_name=@C3,
    last_name=@C4,
    gender=@C5,
    hire_date=@C6;
```

10.186.61.162:3306 demo SQL > select * from emp;

emp_no	birth_date	first_name	last_name	fullname	gender	hire_date	modify_date	delete_flag
10001	1953-09-02	Facello	Georgi	NULL	M	1986-06-26	NULL	NULL
10002	1964-06-02	Simmel	Bezalel	NULL	F	1985-11-21	NULL	NULL
10003	1959-12-03	Bamford	Parto	NULL	M	1986-08-28	NULL	NULL
10004	1954-05-01	Koblick	Chirstian	NULL	M	1986-12-01	NULL	NULL
10005	1955-01-21	Maliniak	Kyoichi	NULL	M	1989-09-12	NULL	NULL
10006	1953-04-20	Preusig	Anneke	NULL	F	1989-06-02	NULL	NULL
10007	1957-05-23	Zielinski	Tzvetan	NULL	F	1989-02-10	NULL	NULL
10008	1958-02-19	Kalloufi	Saniya	NULL	M	1994-09-15	NULL	NULL
10009	1952-04-19	Peac	Sumant	NULL	F	1985-02-18	NULL	NULL
10010	1963-06-01	Piveteau	Duangkaew	NULL	F	1989-08-24	NULL	NULL

10 rows in set (0.0013 sec)

场景 3. LOAD 生成自定义字段数据

从场景 2 的验证可以看到, emp 表中新增的字段 `fullname, modify_date, delete_flag` 字段在导入时并未做处理, 被置为了 NULL 值, 如果需要对其进行处理, 可在 LOAD 时通过 MySQL 支持的函数或给定固定值自行定义数据, 对于文件中存在的字段也可做函数处理, 结合导入导出, 实现简单的 ETL 功能, 如下所示:

```
-- 导入数据语句
load data infile '/data/mysql/3306/tmp/employees.txt'
replace into table demo.emp
character set utf8mb4
fields terminated by ','
enclosed by '"'
lines terminated by '\n'
(@C1,@C2,@C3,@C4,@C5,@C6) -- 该部分对应 employees.txt 文件中 6 列数据

-- 以下部分明确对表中字段与数据文件中的字段做 Mapping 关系,不存在的数据通过函数处理生成(也可设置
```

为固定值)

```
set emp_no=@C1,
    birth_date=@C2,
    first_name=upper(@C3),          -- 将导入的数据转为大写
    last_name=lower(@C4),          -- 将导入的数据转为小写
    fullname=concat(first_name,' ',last_name),    -- 对 first_name 和 last_name 做拼接
    gender=@C5,
    hire_date=@C6 ,
    modify_date=now(),              -- 生成当前时间数据
    delete_flag=if(hire_date<'1988-01-01','Y','N'); -- 对需要生成的值基于某一列做条件运算
```

10.186.61.162:3306 demo SQL > select * from emp;

emp_no	birth_date	first_name	last_name	fullname	gender	hire_date	modify_date	delete_flag
10001	1953-09-02	GEORGI	facello	GEORGI facello	M	1986-06-26	2021-03-08 15:38:34	Y
10002	1964-06-02	BEZALEL	simmel	BEZALEL simmel	F	1985-11-21	2021-03-08 15:38:34	Y
10003	1959-12-03	PARTO	bamford	PARTO bamford	M	1986-08-28	2021-03-08 15:38:34	Y
10004	1954-05-01	CHRISTIAN	koblick	CHRISTIAN koblick	M	1986-12-01	2021-03-08 15:38:34	Y
10005	1955-01-21	KYOICHI	maliniak	KYOICHI maliniak	M	1989-09-12	2021-03-08 15:38:34	N
10006	1953-04-20	ANNEKE	preusig	ANNEKE preusig	F	1989-06-02	2021-03-08 15:38:34	N
10007	1957-05-23	TZVETAN	zielinski	TZVETAN zielinski	F	1989-02-10	2021-03-08 15:38:34	N
10008	1958-02-19	SANIYA	kalloufi	SANIYA kalloufi	M	1994-09-15	2021-03-08 15:38:34	N
10009	1952-04-19	SUMANT	peac	SUMANT peac	F	1985-02-18	2021-03-08 15:38:34	Y
10010	1963-06-01	DUANGKAEW	piveteau	DUANGKAEW piveteau	F	1989-08-24	2021-03-08 15:38:34	N

场景 4. LOAD 定长数据

定长数据的特点如下所示，可以使用函数取出字符串中固定长度来生成指定列数据

```
SQL > select
    c1 as sample_data,
    substr(c1,1,3) as c1,
    substr(c1,4,3) as c2,
    substr(c1,7,2) as c3,
    substr(c1,9,5) as c4,
    substr(c1,14,3) as c5,
    substr(c1,17,3) as c6 from t1

***** 1. row *****
sample_data: ABC 余振兴 CDMYSQLLEFG 数据库
    c1: ABC
    c2: 余振兴
    c3: CD
    c4: MySQL
    c5: EFG
    c6: 数据库
```

定长数据导入需要明确每列数据占用的字符个数，以下直接使用 `rpadd` 对现有的表数据填充空格的方式生成定长数据用作示例使用

-- 生成定长数据

SQL > select

```
concat(rpad(emp_no,10,' '),
       rpad(birth_date,19,' '),
       rpad(first_name,14,' '),
       rpad(last_name,16,' '),
       rpad(gender,2,' '),
       rpad(hire_date,19,' ')) as fixed_length_data
from employees.employees limit 10;
```

```
+-----+
| fixed_length_data |
+-----+
| 10001    1953-09-02      Georgi    Facello      M 1986-06-26 |
| 10002    1964-06-02      Bezalel    Simmel      F 1985-11-21 |
| 10003    1959-12-03      Parto      Bamford     M 1986-08-28 |
| 10004    1954-05-01      Chirstian  Koblick     M 1986-12-01 |
| 10005    1955-01-21      Kyoichi    Maliniak    M 1989-09-12 |
| 10006    1953-04-20      Anneke     Preusig     F 1989-06-02 |
| 10007    1957-05-23      Tzvetan    Zielinski   F 1989-02-10 |
| 10008    1958-02-19      Saniya     Kalloufi    M 1994-09-15 |
| 10009    1952-04-19      Sumant     Peac        F 1985-02-18 |
| 10010    1963-06-01      Duangkaew Piveteau    F 1989-08-24 |
+-----+
```

-- 导出定长数据

select

```
concat(rpad(emp_no,10,' '),
       rpad(birth_date,19,' '),
       rpad(first_name,14,' '),
       rpad(last_name,16,' '),
       rpad(gender,2,' '),
       rpad(hire_date,19,' ')) as fixed_length_data
```

into outfile '/data/mysql/3306/tmp/employees_fixed.txt'

character set utf8mb4

lines terminated by '\n'

from employees.employees limit 10;

-- 导出数据示例

[root@10-186-61-162 tmp]# cat employees_fixed.txt

```
10001    1953-09-02      Georgi    Facello      M 1986-06-26
10002    1964-06-02      Bezalel    Simmel      F 1985-11-21
10003    1959-12-03      Parto      Bamford     M 1986-08-28
10004    1954-05-01      Chirstian  Koblick     M 1986-12-01
```


10005	1955-01-21	Kyoichi	Maliniak	M	1989-09-12
10006	1953-04-20	Anneke	Preusig	F	1989-06-02
10007	1957-05-23	Tzvetan	Zielinski	F	1989-02-10
10008	1958-02-19	Saniya	Kalloufi	M	1994-09-15
10009	1952-04-19	Sumant	Peac	F	1985-02-18
10010	1963-06-01	Duangkaew	Piveteau	F	1989-08-24

```
-- 导入定长数据
load data infile '/data/mysql/3306/tmp/employees_fixed.txt'
replace into table demo.emp
character set utf8mb4
fields terminated by ','
enclosed by ''
lines terminated by '\n'
(@row) -- 对一行数据定义为一个整体
set emp_no = trim(substr(@row,1,10)),-- 使用 substr 取前 10 个字符,并去除头尾空格数据
    birth_date = trim(substr(@row,11,19)),-- 后续字段以此类推
    first_name = trim(substr(@row,30,14)),
    last_name = trim(substr(@row,44,16)),
    fullname = concat(first_name,' ',last_name), -- 对 first_name 和 last_name 做拼接
    gender = trim(substr(@row,60,2)),
    hire_date = trim(substr(@row,62,19)),
    modify_date = now(),
    delete_flag = if(hire_date<'1988-01-01','Y','N'); -- 对需要生成的值基于某一列做条件运算
```

10.186.61.162:3306 demo SQL > select * from emp;

emp_no	birth_date	first_name	last_name	fullname	gender	hire_date	modify_date	delete_flag
10001	1953-09-02	Georgi	Facello	Georgi Facello	M	1986-06-26	2021-03-08 16:59:06	Y
10002	1964-06-02	Bezalel	Simmel	Bezalel Simmel	F	1985-11-21	2021-03-08 16:59:06	Y
10003	1959-12-03	Parto	Bamford	Parto Bamford	M	1986-08-28	2021-03-08 16:59:06	Y
10004	1954-05-01	Chirstian	Koblick	Chirstian Koblick	M	1986-12-01	2021-03-08 16:59:06	Y
10005	1955-01-21	Kyoichi	Maliniak	Kyoichi Maliniak	M	1989-09-12	2021-03-08 16:59:06	N
10006	1953-04-20	Anneke	Preusig	Anneke Preusig	F	1989-06-02	2021-03-08 16:59:06	N
10007	1957-05-23	Tzvetan	Zielinski	Tzvetan Zielinski	F	1989-02-10	2021-03-08 16:59:06	N
10008	1958-02-19	Saniya	Kalloufi	Saniya Kalloufi	M	1994-09-15	2021-03-08 16:59:06	N
10009	1952-04-19	Sumant	Peac	Sumant Peac	F	1985-02-18	2021-03-08 16:59:06	Y
10010	1963-06-01	Duangkaew	Piveteau	Duangkaew Piveteau	F	1989-08-24	2021-03-08 16:59:06	N

五、LOAD 总结

1. 默认情况下导入的顺序以文本文件 列-从左到右, 行-从上到下 的顺序导入
2. 如果表结构和文本数据不一致, 建议将文本文件中的各列依次顺序编号并与表中字段建立 mapping 关系, 以防数据导入到错误的字段
3. 对于待导入的文本文件较大的场景, 建议将文件 按行拆分 为多个小文件, 如用 split 拆分

4. 对文件导入后建议执行以下语句验证导入的数据是否有 `Warning` , `ERROR` 以及导入的数据量

- `GET DIAGNOSTICS @p1=NUMBER,@p2=ROW_COUNT;`
- `select @p1 AS ERROR_COUNT,@p2 as ROW_COUNT;`

5. 文本文件数据与表结构存在过大的差异或数据需要做清洗转换, 建议还是用专业的 ETL 工具或先粗略导入 MySQL 中再进行加工转换处理。

3.10 MySQL Shell import_table 数据导入

作者: 余振兴

1. import_table 介绍

上期技术分享我介绍了 MySQL Load Data 的 4 种将文本数据导入到 MySQL 的常用的方法。

这一期我们继续介绍另一款更加高效的数据导入工具 —— MySQL Shell 工具集中的 `import_table` 。该工具的全称是 `Parallel Table Import Utility` , 顾名思义, 支持并发数据导入。

该工具在 MySQL Shell 8.0.23 版本后, 功能更加完善, 以下列举该工具的核心功能:

- 基本覆盖了 MySQL Data Load 的所有功能, 可以作为替代品使用
- 默认支持并发导入 (支持自定义 `chunk` 大小)
- 支持通配符匹配多个文件同时导入到一张表 (非常适用于相同结构数据汇总到一张表)
- 支持限速 (对带宽使用有要求的场景, 非常合适)
- 支持对压缩文件处理
- 支持导入到 5.7 及以上 MySQL

2. Load Data 与 import_table 功能示例

该部分针对 `import_table` 和 `Load Data` 相同的功能做命令示例演示, 我们依旧以导入 `employees` 表的示例数据为例, 演示 MySQL Load Data 的综合场景。

- 数据自定义顺序导入
- 数据函数处理
- 自定义数据取值

示例数据如下

```
[root@10-186-61-162 tmp]# cat employees_01.csv
"10001","1953-09-02","Georgi","Facello","M","1986-06-26"
"10003","1959-12-03","Parto","Bamford","M","1986-08-28"
"10002","1964-06-02","Bezalel","Simmel","F","1985-11-21"
"10004","1954-05-01","Chirstian","Koblick","M","1986-12-01"
```

```
"10005","1955-01-21","Kyoichi","Maliniak","M","1989-09-12"
"10006","1953-04-20","Anneke","Preusig","F","1989-06-02"
"10007","1957-05-23","Tzvetan","Zielinski","F","1989-02-10"
"10008","1958-02-19","Saniya","Kalloufi","M","1994-09-15"
"10009","1952-04-19","Sumant","Peac","F","1985-02-18"
"10010","1963-06-01","Duangkaew","Piveteau","F","1989-08-24"
```

示例表结构

```
10.186.61.162:3306 employees SQL > desc emp;
```

Field	Type	Null	Key	Default	Extra	
emp_no	int	NO	PRI	NULL		
birth_date	date	NO		NULL		
first_name	varchar(14)	NO		NULL		
last_name	varchar(16)	NO		NULL		
full_name	varchar(64)	YES		NULL		-- 表新增字段,导出数据文件中不存在
gender	enum('M','F')	NO		NULL		
hire_date	date	NO		NULL		
modify_date	datetime	YES		NULL		-- 表新增字段,导出数据文件中不存在
delete_flag	varchar(1)	YES		NULL		-- 表新增字段,导出数据文件中不存在

2.1 用 Load Data 方式导入数据

具体参数含义不做说明,需要了解语法规则及含义可查看系列上一篇文章《MySQL Load Data 的多种用法》

```
load data infile '/data/mysql/3306/tmp/employees_01.csv'
into table employees.emp
character set utf8mb4
fields terminated by ','
enclosed by '"'
lines terminated by '\n'
(@C1,@C2,@C3,@C4,@C5,@C6)
set emp_no=@C1,
    birth_date=@C2,
    first_name=upper(@C3),
    last_name=lower(@C4),
    full_name=concat(first_name,' ',last_name),
    gender=@C5,
```

```
hire_date=@C6 ,
modify_date=now(),
delete_flag=if(hire_date<'1988-01-01','Y','N');
```

```
10.186.61.162:3306 employees SQL > load data infile '/data/mysql/3306/tmp/employees_01.csv'
-> into table employees.emp
-> character set utf8mb4
-> fields terminated by ','
-> enclosed by '"'
-> lines terminated by '\n'
-> (@C1,@C2,@C3,@C4,@C5,@C6)
-> set emp_no=@C1,
-> birth_date=@C2,
-> first_name=upper(@C3),
-> last_name=lower(@C4),
-> full_name=concat(first_name,' ',last_name),
-> gender=@C5,
-> hire_date=@C6 ,
-> modify_date=now(),
-> delete_flag=if(hire_date<'1988-01-01','Y','N');
```

Query OK, 10 rows affected (0.0045 sec)

Records: 10 Deleted: 0 Skipped: 0 Warnings: 0

```
10.186.61.162:3306 employees SQL > select * from emp;
```

emp_no	birth_date	first_name	last_name	full_name	gender	hire_date	modify_date	delete_flag
10001	1953-09-02	GEORGI	facello	GEORGI facello	M	1986-06-26	2021-03-29 10:54:35	Y
10002	1964-06-02	BEZALEL	simmel	BEZALEL simmel	F	1985-11-21	2021-03-29 10:54:35	Y
10003	1959-12-03	PARTO	bamford	PARTO bamford	M	1986-08-28	2021-03-29 10:54:35	Y
10004	1954-05-01	CHRISTIAN	koblick	CHRISTIAN koblick	M	1986-12-01	2021-03-29 10:54:35	Y
10005	1955-01-21	KYOICHI	maliniak	KYOICHI maliniak	M	1989-09-12	2021-03-29 10:54:35	N
10006	1953-04-20	ANNEKE	preusig	ANNEKE preusig	F	1989-06-02	2021-03-29 10:54:35	N
10007	1957-05-23	TZVETAN	zielinski	TZVETAN zielinski	F	1989-02-10	2021-03-29 10:54:35	N
10008	1958-02-19	SANIYA	kalloufi	SANIYA kalloufi	M	1994-09-15	2021-03-29 10:54:35	N
10009	1952-04-19	SUMANT	peac	SUMANT peac	F	1985-02-18	2021-03-29 10:54:35	Y
10010	1963-06-01	DUANGKAEW	piveteau	DUANGKAEW piveteau	F	1989-08-24	2021-03-29 10:54:35	N

10 rows in set (0.0008 sec)

2.2 用 import_table 方式导入数据

```
util.import_table(
    [
        "/data/mysql/3306/tmp/employees_01.csv",
    ],
    {
        "schema": "employees",
        "table": "emp",
        "dialect": "csv-unix",
        "skipRows": 0,
        "showProgress": True,
        "characterSet": "utf8mb4",
        "columns": [1,2,3,4,5,6], ## 文件中多少个列就用多少个序号标识就
行
        "decodeColumns": {
            "emp_no": "@1", ## 对应文件中的第 1 列
            "birth_date": "@2", ## 对应文件中的第 2 个列
            "first_name": "upper(@3)", ## 对应文件中的第 3 个列,并做转为大写的
处理
            "last_name": "lower(@4)", ## 对应文件中的第 4 个列,并做转为大写的
```

处理

```
"full_name":      "concat(@3,' ',@4)",      ## 将文件中的第 3,4 列合并成一列生成表
中字段值
"gender":          "@5",                    ## 对应文件中的第 5 个列
"hire_date":       "@6",                    ## 对应文件中的第 6 个列
"modify_date":     "now()",                  ## 用函数生成表中字段值
"delete_flag":     "if(@6<'1988-01-01','Y','N')", ## 基于文件中第 6 列做逻辑判断,
生成表中对应字段值
    }
})
```

```
10.186.61.162:3306 employees Py > util.import_table(
-> [
->     "/data/mysql/3306/tmp/employees_01.csv",
-> ],
-> {
->     "schema": "employees",
->     "table": "emp",
->     "dialect": "csv-unix",
->     "skipRows": 0,
->     "showProgress": True,
->     "characterSet": "utf8mb4",
->     "columns": [1,2,3,4,5,6],          ## 文件中多少个列就用多少个序号标识就行
->     "decodeColumns": {
->         "emp_no": "@1",                ## 对应文件中的第 1 列
->         "birth_date": "@2",            ## 对应文件中的第 2 个列
->         "first_name": "upper(@3)",      ## 对应文件中的第 3 个列,并做转为大写的处理
->         "last_name": "lower(@4)",       ## 对应文件中的第 4 个列,并做转为大写的处理
->         "full_name": "concat(@3,' ',@4)", ## 将文件中的第 3,4 列合并成一列生成表中字段值
->         "gender": "@5",                 ## 对应文件中的第 5 个列
->         "hire_date": "@6",              ## 对应文件中的第 6 个列
->         "modify_date": "now()",          ## 用函数生成表中字段值
->         "delete_flag": "if(@6<'1988-01-01','Y','N')", ## 基于文件中第 6 列做逻辑判断,生成表中对应字段值
->     }
-> })

Importing from file '/data/mysql/3306/tmp/employees_01.csv' to table `employees`.`emp` in MySQL Server at 10.186.61.162:3306 using 1 thread
[Worker000] employees_01.csv: Records: 10 Deleted: 0 Skipped: 0 Warnings: 0
100% (579 bytes / 579 bytes), 0.00 B/s
File '/data/mysql/3306/tmp/employees_01.csv' (579 bytes) was imported in 0.0317 sec at 579.00 B/s
Total rows affected in employees.emp: Records: 10 Deleted: 0 Skipped: 0 Warnings: 0
```

```
10.186.61.162:3306 employees Py > \sql
Switching to SQL mode... Commands end with ;
10.186.61.162:3306 employees SQL > select * from emp;
```

emp_no	birth_date	first_name	last_name	full_name	gender	hire_date	modify_date	delete_flag
10001	1953-09-02	GEORGI	facello	Georgi Facello	M	1986-06-26	2021-03-29 10:59:16	Y
10002	1964-06-02	BEZALEL	simmel	Bezalel Simmel	F	1985-11-21	2021-03-29 10:59:16	Y
10003	1959-12-03	PARTO	bamford	Parto Bamford	M	1986-08-28	2021-03-29 10:59:16	Y
10004	1954-05-01	CHIRSTIAN	koblick	Chirstian Koblick	M	1986-12-01	2021-03-29 10:59:16	Y
10005	1955-01-21	KYOICHI	maliniak	Kyoichi Maliniak	M	1989-09-12	2021-03-29 10:59:16	N
10006	1953-04-20	ANNEKE	preusig	Anneke Preusig	F	1989-06-02	2021-03-29 10:59:16	N
10007	1957-05-23	TZVETAN	zielinski	Tzvetan Zielinski	F	1989-02-10	2021-03-29 10:59:16	N
10008	1958-02-19	SANIYA	kalloufi	Saniya Kalloufi	M	1994-09-15	2021-03-29 10:59:16	N
10009	1952-04-19	SUMANT	peac	Sumant Peac	F	1985-02-18	2021-03-29 10:59:16	Y
10010	1963-06-01	DUANGKAEW	piveteau	Duangkaew Piveteau	F	1989-08-24	2021-03-29 10:59:16	N

```
10 rows in set (0.0009 sec)
```

3. import_table 特定功能

3.1 多文件导入 (模糊匹配)

在导入前我生成好了 3 份单独的 employees 文件,导出的结构一致

```
[root@10-186-61-162 tmp]# ls -lh
```

总用量 1.9G

```
-rw-r----- 1 mysql mysql 579 3月 24 19:07 employees_01.csv
-rw-r----- 1 mysql mysql 584 3月 24 18:48 employees_02.csv
-rw-r----- 1 mysql mysql 576 3月 24 18:48 employees_03.csv
-rw-r----- 1 mysql mysql 1.9G 3月 26 17:15 sbtest1.csv
```

导入命令,其中针对文件用 employees_*做模糊匹配

```
util.import_table(
```

```
    [
        "/data/mysql/3306/tmp/employees_*",
```

```
    ],
```

```
    {
```

```
        "schema": "employees",
```

```
        "table": "emp",
```

```
        "dialect": "csv-unix",
```

```
        "skipRows": 0,
```

```
        "showProgress": True,
```

```
        "characterSet": "utf8mb4",
```

```
        "columns": [1,2,3,4,5,6],
```

文件中多少个列就用多少个序号标识就

行

```
        "decodeColumns": {
```

```
            "emp_no": "@1",
```

对应文件中的第 1 列

```
            "birth_date": "@2",
```

对应文件中的第 2 个列

```
            "first_name": "upper(@3)",
```

对应文件中的第 3 个列,并做转为大写的

处理

```
            "last_name": "lower(@4)",
```

对应文件中的第 4 个列,并做转为大写的

处理

```
            "full_name": "concat(@3, ' ', @4)",
```

将文件中的第 3,4 列合并成一列生成表

中字段值

```
            "gender": "@5",
```

对应文件中的第 5 个列

```
            "hire_date": "@6",
```

对应文件中的第 6 个列

```
            "modify_date": "now()",
```

用函数生成表中字段值

```
            "delete_flag": "if(@6<'1988-01-01','Y','N')" ## 基于文件中第 6 列做逻辑判断,
```

生成表中对应字段值

```
        }
```

```
    })
```

导入命令,其中对要导入的文件均明确指定其路径

```
util.import_table(
```

```
    [
```

```
        "/data/mysql/3306/tmp/employees_01.csv",
```

```
        "/data/mysql/3306/tmp/employees_02.csv",
```

```
        "/data/mysql/3306/tmp/employees_03.csv"
```



```

],
{
    "schema": "employees",
    "table": "emp",
    "dialect": "csv-unix",
    "skipRows": 0,
    "showProgress": True,
    "characterSet": "utf8mb4",
    "columns": [1,2,3,4,5,6],          ## 文件中多少个列就用多少个序号标识就
行
    "decodeColumns": {
        "emp_no":      "@1",          ## 对应文件中的第 1 列
        "birth_date":  "@2",          ## 对应文件中的第 2 个列
        "first_name":  "upper(@3)",   ## 对应文件中的第 3 个列,并做转为大写的
处理
        "last_name":   "lower(@4)",   ## 对应文件中的第 4 个列,并做转为大写的
处理
        "full_name":   "concat(@3, ' ', @4)", ## 将文件中的第 3,4 列合并成一列生成表
中字段值
        "gender":      "@5",          ## 对应文件中的第 5 个列
        "hire_date":   "@6",          ## 对应文件中的第 6 个列
        "modify_date": "now()",       ## 用函数生成表中字段值
        "delete_flag": "if(@6<'1988-01-01','Y','N')" ## 基于文件中第 6 列做逻辑判断,
生成表中对应字段值
    }
})

```

```

10.186.61.162:3306 employees Py > util.import_table(
-> {
->     "/data/mysql/3306/tmp/employees_*.csv",
-> },
-> {
->     "schema": "employees",
->     "table": "emp",
->     "dialect": "csv-unix",
->     "skipRows": 0,
->     "showProgress": True,
->     "characterSet": "utf8mb4",
->     "columns": [1,2,3,4,5,6],          ## 文件中多少个列就用多少个序号标识就
->     "decodeColumns": {
->         "emp_no":      "@1",          ## 对应文件中的第 1 列
->         "birth_date":  "@2",          ## 对应文件中的第 2 个列
->         "first_name":  "upper(@3)",   ## 对应文件中的第 3 个列,并做转为大写的处理
->         "last_name":   "lower(@4)",   ## 对应文件中的第 4 个列,并做转为大写的处理
->         "full_name":   "concat(@3, ' ', @4)", ## 将文件中的第 3,4 列合并成一列生成表中字段值
->         "gender":      "@5",          ## 对应文件中的第 5 个列
->         "hire_date":   "@6",          ## 对应文件中的第 6 个列
->         "modify_date": "now()",       ## 用函数生成表中字段值
->         "delete_flag": "if(@6<'1988-01-01','Y','N')" ## 基于文件中第 6 列做逻辑判断,生成表中对应字段值
->     }
-> })
Importing from multiple files to table 'employees`.`emp` in MySQL Server at 10.186.61.162:3306 using 8 threads
[Worker000] employees_01.csv: Records: 10 Deleted: 0 Skipped: 0 Warnings: 0
[Worker000] employees_02.csv: Records: 10 Deleted: 0 Skipped: 0 Warnings: 0
[Worker000] employees_03.csv: Records: 10 Deleted: 0 Skipped: 0 Warnings: 0
100% (1.74 KB / 1.74 KB), 0.00 B/s
3 file(s) (1.74 KB) was imported in 0.0742 sec at 1.74 KB/s
Total rows affected in employees.emp: Records: 30 Deleted: 0 Skipped: 0 Warnings: 0
10.186.61.162:3306 employees Py >

```

```

10.186.61.162:3306 employees Py > util.import_table(
-> [
->     "/data/mysql/3306/tmp/employees_01.csv",
->     "/data/mysql/3306/tmp/employees_02.csv",
->     "/data/mysql/3306/tmp/employees_03.csv"
-> ],
-> {
->     "schema": "employees",
->     "table": "emp",
->     "dialect": "csv-unix",
->     "skipRows": 0,
->     "showProgress": True,
->     "characterSet": "utf8mb4",
->     "columns": [1,2,3,4,5,6],
->     "decodeColumns": {
->         "emp_no": "@1",
->         "birth_date": "@2",
->         "first_name": "upper(@3)",
->         "last_name": "lower(@4)",
->         "full_name": "concat(@3, ' ', @4)",
->         "gender": "@5",
->         "hire_date": "@6",
->         "modify_date": "now()",
->         "delete_flag": "if(@6<'1988-01-01','Y','N')"
->     }
-> })
Importing from multiple files to table `employees`.`emp` in MySQL Server at 10.186.61.162:3306 using 8 threads
[Worker007] employees_01.csv: Records: 10 Deleted: 0 Skipped: 0 Warnings: 0
[Worker007] employees_03.csv: Records: 10 Deleted: 0 Skipped: 0 Warnings: 0
[Worker005] employees_02.csv: Records: 10 Deleted: 0 Skipped: 0 Warnings: 0
100% (1.74 KB / 1.74 KB), 0.00 B/s
3 file(s) (1.74 KB) was imported in 0.0604 sec at 1.74 KB/s
Total rows affected in employees.emp: Records: 30 Deleted: 0 Skipped: 0 Warnings: 0

```

3.2 并发导入

在实验并发导入前我们创建一张 1000W 的 sbtest1 表（大约 2G 数据），做并发模拟，import_table 用 threads 参数作为并发配置，默认为 8 个并发。

导出测试需要的 sbtest1 数据

```
[root@10-186-61-162 tmp]# ls -lh
```

总用量 1.9G

```

-rw-r----- 1 mysql mysql 579 3月 24 19:07 employees_01.csv
-rw-r----- 1 mysql mysql 584 3月 24 18:48 employees_02.csv
-rw-r----- 1 mysql mysql 576 3月 24 18:48 employees_03.csv
-rw-r----- 1 mysql mysql 1.9G 3月 26 17:15 sbtest1.csv

```

开启 threads 为 8 个并发

```

util.import_table(
    [
        "/data/mysql/3306/tmp/sbtest1.csv",
    ],
    {
        "schema": "demo",
        "table": "sbtest1",
        "dialect": "csv-unix",
        "skipRows": 0,
        "showProgress": True,
        "characterSet": "utf8mb4",
        "threads": "8"
    })

```



```

10.186.61.162:3306 employees Py > util.import_table(
    -> [
    ->     "/data/mysql/3306/tmp/sbtest1.csv",
    -> ],
    -> {
    ->     "schema": "demo",
    ->     "table": "sbtest1",
    ->     "dialect": "csv-unix",
    ->     "skipRows": 0,
    ->     "showProgress": True,
    ->     "characterSet": "utf8mb4",
    ->     "threads": "8"
    -> })
Importing from file '/data/mysql/3306/tmp/sbtest1.csv' to table `demo`.`sbtest1` in MySQL Server at 10.186.61.162:3306 using 8 threads
[Worker000] sbtest1.csv: Records: 246853 Deleted: 0 Skipped: 0 Warnings: 0
[Worker005] sbtest1.csv: Records: 245099 Deleted: 0 Skipped: 0 Warnings: 0
[Worker004] sbtest1.csv: Records: 245099 Deleted: 0 Skipped: 0 Warnings: 0
[Worker002] sbtest1.csv: Records: 246306 Deleted: 0 Skipped: 0 Warnings: 0
[Worker006] sbtest1.csv: Records: 245168 Deleted: 0 Skipped: 0 Warnings: 0
[Worker003] sbtest1.csv: Records: 246306 Deleted: 0 Skipped: 0 Warnings: 0
25% (523.73 MB / 2.04 GB), 20.14 MB/s
[Worker004] sbtest1.csv: Records: 245099 Deleted: 0 Skipped: 0 Warnings: 0
[Worker002] sbtest1.csv: Records: 245099 Deleted: 0 Skipped: 0 Warnings: 0
[Worker005] sbtest1.csv: Records: 245099 Deleted: 0 Skipped: 0 Warnings: 0
[Worker000] sbtest1.csv: Records: 245099 Deleted: 0 Skipped: 0 Warnings: 0
[Worker006] sbtest1.csv: Records: 245099 Deleted: 0 Skipped: 0 Warnings: 0
[Worker003] sbtest1.csv: Records: 245099 Deleted: 0 Skipped: 0 Warnings: 0
[Worker004] sbtest1.csv: Records: 190596 Deleted: 0 Skipped: 0 Warnings: 0
[Worker001] sbtest1.csv: Records: 245099 Deleted: 0 Skipped: 0 Warnings: 0
[Worker007] sbtest1.csv: Records: 245099 Deleted: 0 Skipped: 0 Warnings: 0
100% (2.04 GB / 2.04 GB), 1.22 MB/s
File '/data/mysql/3306/tmp/sbtest1.csv' (2.04 GB) was imported in 2 min 56.3347 sec at 11.56 MB/s
Total rows affected in demo.sbtest1: Records: 10000000 Deleted: 0 Skipped: 0 Warnings: 0

```

3.3 导入速率控制

可以通过 `maxRate` 和 `threads` 来控制每个并发线程的导入数据。如，当前配置线程为 4 个，每个线程的速率为 2M/s，则最高不会超过 8M/s。

```

util.import_table(
    [
        "/data/mysql/3306/tmp/sbtest1.csv",
    ],
    {
        "schema": "demo",
        "table": "sbtest1",
        "dialect": "csv-unix",
        "skipRows": 0,
        "showProgress": True,
        "characterSet": "utf8mb4",
        "threads": "4",
        "maxRate": "2M"
    })

```

```

10.186.61.162:3306 demo Py > util.import_table(
    [
        "/data/mysql/3306/tmp/sbtest1.csv",
    ],
    {
        "schema": "demo",
        "table": "sbtest1",
        "dialect": "csv-unix",
        "skipRows": 0,
        "showProgress": True,
        "characterSet": "utf8mb4",
        "threads": "4",
        "maxRate": "2M"
    })
Importing from file '/data/mysql/3306/tmp/sbtest1.csv' to table `demo`.`sbtest1` in MySQL Server at 10.186.61.162:3306 using 4 threads
[Worker002] sbtest1.csv: Records: 246306 Deleted: 0 Skipped: 0 Warnings: 0
[Worker003] sbtest1.csv: Records: 246306 Deleted: 0 Skipped: 0 Warnings: 0
[Worker001] sbtest1.csv: Records: 246306 Deleted: 0 Skipped: 0 Warnings: 0
[Worker000] sbtest1.csv: Records: 246853 Deleted: 0 Skipped: 0 Warnings: 0
12% (262.91 MB / 2.04 GB), 7.67 MB/s

```

3.4 自定义 chunk 大小

默认的 chunk 大小为 50M，我们可以调整 chunk 的大小，减少事务大小，如我们将 chunk 大小调整为 1M，则每个线程每次导入的数据量也相应减少。

```

util.import_table(
    [
        "/data/mysql/3306/tmp/sbtest1.csv",
    ],
    {
        "schema": "demo",
        "table": "sbtest1",
        "dialect": "csv-unix",
        "skipRows": 0,
        "showProgress": True,
        "characterSet": "utf8mb4",
        "threads": "4",
        "bytesPerChunk": "1M",
        "maxRate": "2M"
    })

```

```

10.186.61.162:3306 demo Py > util.import_table(
    [
        "/data/mysql/3306/tmp/sbtest1.csv",
    ],
    {
        "schema": "demo",
        "table": "sbtest1",
        "dialect": "csv-unix",
        "skipRows": 0,
        "showProgress": True,
        "characterSet": "utf8mb4",
        "threads": "4",
        "bytesPerChunk": "1M",
        "maxRate": "2M"
    })
Importing from file '/data/mysql/3306/tmp/sbtest1.csv' to table `demo`.`sbtest1` in MySQL Server at 10.186.61.162:3306 using 4 threads
[Worker001] sbtest1.csv: Records: 4976 Deleted: 0 Skipped: 0 Warnings: 0
[Worker000] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker002] sbtest1.csv: Records: 4981 Deleted: 0 Skipped: 0 Warnings: 0
[Worker003] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker000] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker001] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker002] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker003] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker000] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker001] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker000] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker003] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker000] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker002] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker001] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker003] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0
[Worker002] sbtest1.csv: Records: 4951 Deleted: 0 Skipped: 0 Warnings: 0

```

4. Load Data vs import_table 性能对比

- 使用相同库表
- 不对数据做特殊处理，原样导入
- 不修改参数默认值，只指定必备参数

```

-- Load Data 语句
load data infile '/data/mysql/3306/tmp/sbtest1.csv'
into table demo.sbtest1
character set utf8mb4
fields terminated by ','
enclosed by '"'
lines terminated by '\n'

-- import_table 语句
util.import_table(
    [
        "/data/mysql/3306/tmp/sbtest1.csv",
    ],
    {
        "schema": "demo",
        "table": "sbtest1",
        "dialect": "csv-unix",
        "skipRows": 0,
        "showProgress": True,
        "characterSet": "utf8mb4"
    })

```

```

10.186.61.162:3306 demo SQL > load data infile '/data/mysql/3306/tmp/sbtest1.csv'
-> into table demo.sbtest1
-> character set utf8mb4
-> fields terminated by ','
-> enclosed by '"'
-> lines terminated by '\n';
^@^@^@Query OK, 10000000 rows affected (4 min 53.0183 sec)

Records: 10000000 Deleted: 0 Skipped: 0 Warnings: 0

```

```

10.186.61.162:3306 demo Py > util.import_table(
-> {
->     "/data/mysql/3306/tmp/sbtest1.csv",
-> },
-> {
->     "schema": "demo",
->     "table": "sbtest1",
->     "dialect": "csv-unix",
->     "skipRows": 0,
->     "showProgress": True,
->     "characterSet": "utf8mb4"
-> })
Importing from file '/data/mysql/3306/tmp/sbtest1.csv' to table 'demo`.`sbtest1' in MySQL Server at 10.186.61.162:3306 using 8 threads
100% [2.04 GB / 2.04 GB], 1.82 MB/s
File '/data/mysql/3306/tmp/sbtest1.csv' [2.04 GB] was imported in 2 min 19.6896 sec at 14.60 MB/s
Total rows affected in demo.sbtest1: Records: 10000000 Deleted: 0 Skipped: 0 Warnings: 0

```

可以看到，Load Data 耗时约 5 分钟，而 import_table 则只要不到一半的时间即可完成数据导入，效率高一倍以上（虚拟机环境磁盘 IO 能力有限情况下）。

5. 技术总结

- import_table 包含了 Load Data 几乎所有的功能
- import_table 导入的效率比 Load Data 更高
- import_table 支持对导入速度，并发以及每次导入的数据大小做精细控制
- import_table 的导入进度报告更加详细，便于排错及时间评估，包括：
 - 导入速度
 - 导入总耗时
 - 每批次导入的数据量，是否存在 Warning 等等
 - 导入最终的汇总报告